

Universitat Politècnica de Catalunya

FACULTAT D'INFORMÀTICA DE BARCELONA

SEMANTIC DATA MANAGEMENT FOR AI

Bases de Dades Avançades

Grau en Intel·ligència Artificial

Joel Alfaro

Pablo Chacón

Albert Roca Pedrós

22/05/2026

Índex

1	Project Scope and Objectives	2
2	Data Engineering Pipeline	2
3	Knowledge Graph Design Decisions	3
4	SPARQL Analysis Pipeline	4
5	KG Embedding and Machine Learning Pipeline	4
6	Visualization and Outputs	5
7	Validation and Limitations	7

1 Project Scope and Objectives

This project extends the previous Barcelona tourism data pipeline with a Knowledge Graph (KG) exploitation zone and two KG-based analytical pipelines. The goal is to move from integrated relational data to a semantic representation that can be queried and analysed through graph concepts, as required in Project 2.

The starting point is the cleaned and integrated DuckDB database generated in Project 1, located by default at `../BDA_DataPipeline/exploitation_zone/exploitation_zone.duckdb`. This repository does not modify the Project 1 pipeline. Instead, it consumes the trusted and exploitation-ready tables from that project and builds a new semantic layer in `exploitation_zone/barcelona_tourism_kg.ttl`. The final deliverables are Python scripts, reusable source modules, generated analytical reports, an embedding CSV file, and a dependency-free HTML dashboard.

The implemented workflow covers the main requirements of the assignment:

- creation of a KG-based exploitation zone using RDF/RDFS;
- a SPARQL pattern-matching analysis pipeline;
- an ML pipeline over graph-derived KG embeddings;
- documentation of the design decisions and non-obvious implementation choices.

2 Data Engineering Pipeline

The Landing, Formatted and Trusted zones are inherited from Project 1. In Project 2, the focus is the semantic exploitation zone. The input data comes from five integrated DuckDB tables: district profiles, neighborhood profiles, neighborhood income profiles, HUT license profiles, and Airbnb zone features. These tables provide the information needed to model Barcelona tourism pressure at neighborhood level.

The KG construction process is implemented in `src/semantic_ai/kg_builder.py` and executed through `scripts/build_kg.py`. The script reads the Project 1 DuckDB database, extracts the relevant integrated tables, normalizes entity labels, creates RDF resources and serializes the final graph in Turtle format. The generated graph currently contains 1153 triples, 10 districts, 73 neighborhoods, 73 Airbnb zones and 3 tourism pressure levels.

The main RDF/RDFS classes are `District`, `Neighborhood`, `AirbnbZone` and `TourismPressureLevel`. The most important properties are `inDistrict`, `hasTourismPressure`, `hasAirbnbZone`, `describesNeighborhood`, `listingCount`, `avgPrice`, `avgRating`, `tourismAssetScore`, `incomeEur`, `hutCount` and `licensedBeds`. The RDFS schema also declares domains and ranges for the main properties, which makes the intended use of each relation explicit and strengthens the semantic model.

The generated KG acts as the new exploitation zone. This is an important design decision: downstream analysis does not query the original relational tables directly, but consumes the RDF graph. Therefore, joins between districts, neighborhoods, income indicators, HUT licenses and Airbnb features are materialized as semantic relationships and literals. This makes the analysis layer less dependent on the physical schema of the DuckDB database and closer to the conceptual domain model.

3 Knowledge Graph Design Decisions

The KG is built at neighborhood granularity because the previous pipeline already produced reliable integrated data at this level. Modeling individual Airbnb listings, hotels or tourism assets as separate nodes would increase graph size, but it would also add complexity that is not necessary for the current analytical goal. The chosen design prioritizes a clear semantic view of tourism pressure across Barcelona neighborhoods.

RDF/RDFS was selected instead of a heavier ontology stack. RDFS classes and properties are sufficient to define the common vocabulary required by the exploitation zone, while keeping the graph easy to inspect, query and extend. The graph is serialized as Turtle because it is human-readable and directly supported by RDFLib and SPARQL tooling.

Neighborhood labels are canonicalized before creating URIs. This avoids duplicate identities when different sources use small spelling variants. For example, **el Poble Sec** and **el Poble-sec** are merged into the same resource, `bda:neighborhood/el-poble-sec`. URI creation also removes accents, lowercases text and replaces non-alphanumeric characters with hyphens. This makes identifiers stable and safe for RDF usage.

Airbnb information is represented through explicit **AirbnbZone** resources instead of storing all Airbnb literals directly on neighborhoods. Each neighborhood is linked to one Airbnb zone using `hasAirbnbZone`, and each zone points back to the neighborhood using `describesNeighborhood`. Listing count, average price and average rating are attached to the Airbnb zone resource. This makes the graph schema more expressive and leaves room for future extensions where Airbnb zones could contain richer accommodation-related attributes.

The RDFS schema declares domains and ranges for the main properties. For example, `inDistrict` links neighborhoods to districts, `hasTourismPressure` links neighborhoods to pressure levels, and Airbnb metrics are attached to **AirbnbZone** resources. The property `tourismAssetScore` is kept with a numeric range but without a single strict domain because it is used both for districts and neighborhoods. This avoids incorrect RDFS inferences while preserving the datatype constraint.

When several rows map to the same canonical neighborhood, numeric metrics are merged by taking the maximum non-null value. This conservative rule was chosen because some source rows may contain missing or zero values while another source row contains the populated metric. Taking the maximum preserves the available tourism signal and avoids underestimating activity due to sparse data.

Tourism pressure is represented as a semantic graph concept rather than only as raw numeric columns. The pressure score combines Airbnb listings, HUT licenses and tourism asset score:

$$score = listings + 1.5 \cdot HUTs + 0.5 \cdot tourismAssets$$

Neighborhoods are classified as **high** when the score is at least 200, **medium** when it is at least 40, and **low** otherwise. This rule is intentionally transparent, so it can be explained and adjusted if the organization wants different policy thresholds.

The weights give more importance to HUT licenses than to Airbnb listing counts because HUTs represent formally licensed tourist accommodation capacity. Tourism assets are also included, but with a lower weight, because they represent attractiveness or concentration of points of interest rather than accommodation pressure itself. The rule is not intended to be a definitive urban policy metric; it is a reproducible semantic feature that allows the graph to expose a meaningful concept for both SPARQL queries and ML embeddings.

4 SPARQL Analysis Pipeline

The first data analysis pipeline uses SPARQL pattern matching over the RDF graph. It is implemented in `src/semantic_ai/sparql_analysis.py` and executed with `scripts/run_sparql_analysis.py`. The pipeline loads the Turtle graph with RDFLib, runs predefined SPARQL queries and exports a Markdown report to `analysis_zone/sparql_analysis.md`.

Three analytical questions were implemented. First, the pipeline identifies high tourism pressure neighborhoods by matching `Neighborhood` resources linked to the `high` pressure level. The top results include la Dreta de l'Eixample, el Raval, Sant Pere, Santa Caterina i la Ribera, el Barri Gotic, la Vila de Gracia and la Sagrada Familia. These areas combine high Airbnb supply, many HUT licenses or high tourism asset concentration.

Second, the pipeline detects neighborhoods with many HUT licenses and low income by filtering neighborhoods with at least 10 HUT licenses and average income below 22,000 EUR. This query is useful because it combines tourism accommodation pressure with a socioeconomic indicator. Examples returned by the query include el Raval, Sant Pere, Santa Caterina i la Ribera, el Barri Gotic, Hostafrancs, la Font de la Guatlla and la Barceloneta.

Third, the pipeline ranks neighborhoods by tourism asset score and Airbnb supply. This helps identify areas where physical tourism attractiveness and short-term rental activity overlap. la Dreta de l'Eixample is the strongest case, with 1,942 listings, 1,802 HUT licenses and the highest tourism asset score in the graph.

The value of using SPARQL is that these questions are expressed in terms of graph concepts instead of table-specific joins. For instance, Airbnb metrics are reached by following `hasAirbnbZone` from a neighborhood to its Airbnb zone, while tourism pressure is reached through `hasTourismPressure`. If more data sources were added later, such as hotels or points of interest, the same query style could be extended by adding new graph properties without changing the analytical interface completely.

5 KG Embedding and Machine Learning Pipeline

The second analysis pipeline builds graph-derived embeddings for each neighborhood and applies clustering. It is implemented in `src/semantic_ai/embedding_ml.py` `scripts/run_embedding_ml.py` `analysis_zone/kg_embedding_clusters.md` `analysis_zone/kg_embedding_clusters.csv`

Instead of using a black-box random-walk method such as Node2Vec, the project uses interpretable semantic embeddings. Each neighborhood vector is extracted from RDF triples and graph relations. The features include direct and related literals such as Airbnb listing count, average price, average rating, tourism asset score, income, HUT licenses and licensed beds. Airbnb features are obtained through the `hasAirbnbZone` relation, which means the ML pipeline consumes the semantic graph model rather than assuming that all literals are attached to the neighborhood node.

The embeddings also include semantic and relational features: ordinal tourism pressure, a high-pressure flag, HUT-to-listing ratio, beds per HUT license, listing share inside the district, income relative to the district average, tourism assets relative to the district average, and a combined low-income high-tourism flag. These features are derived from graph relationships and therefore preserve domain meaning while producing a numerical matrix suitable for machine learning.

Missing income values are not converted directly to zero. Instead, the pipeline marks them with an `incomeMissing` feature and imputes income using the district median when possible, followed by the global median as fallback. This avoids treating unknown socioeconomic data as real zero income and reduces distortion in clustering. In the current generated embeddings, 2 neighborhoods have missing income values and are handled through this controlled imputation process.

This design was chosen because the graph contains 73 neighborhood nodes, which is relatively small for random-walk embeddings. A transparent feature representation is easier to validate, explain and connect to public-policy questions. The embeddings still exploit the KG structure because they use the semantic pressure concept, Airbnb zone relationships and district-relative graph relationships, not only raw table columns.

The standardized embedding matrix is clustered with two methods. KMeans is used as a centroid-based baseline, while Agglomerative Clustering with Ward linkage is used as a hierarchical alternative suited to a small neighborhood dataset. Both methods use four clusters. The KMeans silhouette score is 0.175, while the agglomerative method obtains 0.165. These values indicate that the neighborhoods do not form perfectly separated groups, which is expected in an urban dataset where tourism, income and accommodation indicators change gradually rather than through hard boundaries.

The pipeline also computes a two-dimensional PCA projection for visual inspection. PCA is not used as the clustering model; it is only used to inspect whether the generated groups are separated in a low-dimensional view. The CSV includes 73 neighborhoods: 28 high-pressure, 23 medium-pressure and 22 low-pressure neighborhoods. Nine neighborhoods are marked by the combined low-income and high-tourism-pressure signal.

The clustering results are interpretable because the exported reports include cluster centroids. KMeans separates a small group of extremely high-tourism neighborhoods, including cases with very high listing counts and HUT license counts, from larger groups with medium or low tourism activity. The agglomerative model also isolates strong local patterns, including la Dreta de l'Eixample as an outlier with very high Airbnb supply, HUT licenses and tourism asset score. The purpose of these models is therefore not only predictive performance, but also exploratory segmentation of neighborhoods using semantic KG-derived features.

The low-income high-tourism feature is especially relevant from an analytical perspective. It does not only describe tourism intensity, but also identifies neighborhoods where tourism pressure overlaps with lower income levels. This kind of feature illustrates why the KG representation is useful for AI tasks: semantic concepts can be transformed into numerical features while preserving their domain meaning.

6 Visualization and Outputs

To make the embedding results easier to inspect, the project generates a static HTML dashboard with `scripts/build_embedding_dashboard.py`. The dashboard reads the embedding CSV and produces `analysis_zone/kg_embedding_dashboard.html`. It can be opened directly by loading `analysis_zone/kg_embedding_dashboard.html` in any modern browser, or by serving the `analysis_zone/` folder with a simple local HTTP server. No backend application or database connection is required to inspect the results.

The dashboard shows a PCA scatter plot where each point is a neighborhood. The user can switch the color encoding between KMeans and Agglomerative clusters, select individual neighborhoods, inspect their metrics

and compare cluster summaries. The table below the chart exposes the same values used by the model, which keeps the visualization consistent with the analytical pipeline.

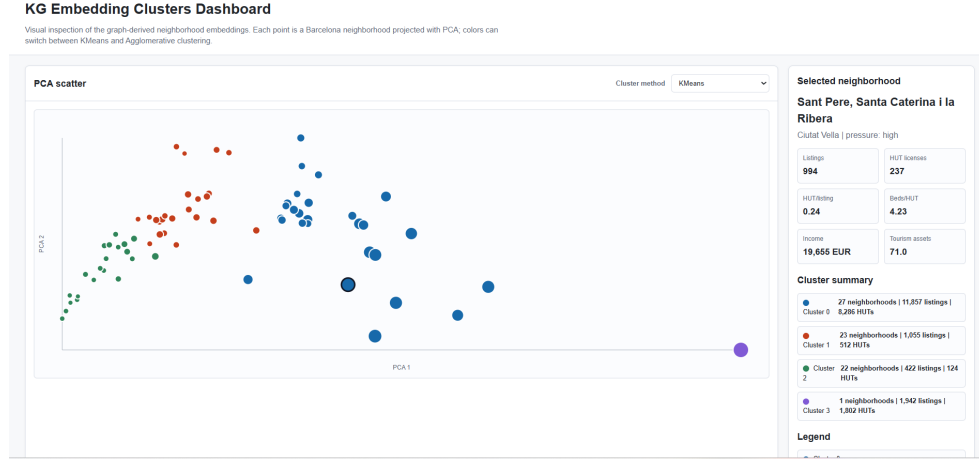


Figura 1: PCA projection of KG-derived neighborhood embeddings, colored by clustering method in the generated dashboard.

Figure 1 shows that the embedding space contains one clearly extreme high-tourism profile and several broader groups with gradual separation, which is consistent with the low silhouette scores reported by the clustering pipeline.

The repository is organized so that the project can be deployed and reproduced through scripts:

- `scripts/build_kg.py`: builds the RDF/Turtle KG from the Project 1 DuckDB database;
- `scripts/run_sparql_analysis.py`: runs SPARQL queries and exports the report;
- `scripts/run_embedding_ml.py`: extracts embeddings, clusters neighborhoods and exports Markdown/CSV outputs;
- `scripts/build_embedding_dashboard.py`: creates the interactive HTML dashboard.

The required Python dependencies are `duckdb`, `rdflib`, `scikit-learn`, `pandas` and `tabulate`. The project can be reproduced by installing `requirements.txt` and running the four scripts in order from the repository root.

The expected execution order is:

1. build the semantic exploitation zone with `python scripts/build_kg.py`;
2. run the graph query analysis with `python scripts/run_sparql_analysis.py`;
3. generate embeddings and clusters with `python scripts/run_embedding_ml.py`;
4. create the visual dashboard with `python scripts/build_embedding_dashboard.py`.

Each step writes its output to either `exploitation_zone/` or `analysis_zone/`, so the pipeline can be rerun from scratch without manual editing.

7 Validation and Limitations

Unit tests are included in the `tests/` directory. They cover URI normalization, literal cleanup, neighborhood alias merging, tourism pressure classification, RDFS domain and range declarations, Airbnb zone creation, embedding feature extraction, clustering output generation and dashboard generation. These tests are important because the most fragile parts of the pipeline are semantic identity creation, schema consistency and the transformation from RDF triples to numerical ML features.

The tests also document expected behaviour. For example, they verify that `el Poble Sec` and `el Poble-sec` are merged into one RDF entity, that each neighborhood can be linked to an `AirbnbZone`, that Airbnb metrics are stored on the Airbnb zone resource, and that semantic embedding columns such as `pressureOrdinal`, `hutListingRatio`, `bedsPerHut` and `incomeMissing` are created. This makes the implementation easier to maintain if the KG schema is extended in future iterations.

The current implementation has some limitations. First, the KG focuses on neighborhood-level summaries and does not yet model individual listings, hotels or tourism assets as first-class entities. Second, the tourism pressure thresholds are transparent but manually defined, so they should be reviewed with domain experts before policy usage. Third, the ML pipeline uses interpretable graph-derived features rather than a learned graph embedding model. This is appropriate for the current graph size, but `Node2Vec`, `RDF2Vec` or graph neural methods could be explored if the graph is extended with richer entity-level relationships.

Another limitation is that some numerical values still depend on the completeness of the original Project 1 data sources. The pipeline now handles missing income values more carefully through explicit missingness indicators and controlled imputation, but the quality of the final analytical features still depends on the coverage and reliability of the source datasets.

Overall, the project satisfies the assignment by creating a semantic exploitation zone and two independent KG-based analysis pipelines. The KG makes the integrated tourism data queryable through a common vocabulary, the SPARQL pipeline extracts interpretable graph patterns, and the ML pipeline groups neighborhoods using semantic features derived from the graph. The latest improvements also make the model semantically stronger by instantiating `AirbnbZone` resources, declaring RDFS domains and ranges, and treating missing numerical values more robustly.